BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

Course: Object Oriented Programming

FINAL PROJECT REPORT

Project title: *Online Learning Platform*

Group Number: *Group 2*

Group Member 1: *Toghrul Mammadli*

Group Member 2: *Aydin Naghiyev*

Group Member 3: *Muhammad Teymurlu*

Report submitted: 10 April 2026

Supervisor: Huseynov Shamistan

Contents

Introduction 2

Architectural Design 2

1. The Model 2

2. The View 2

3. The Controller 3

4. The Repository 4

Object-Oriented Domain Model 4

Database & Entity Relations 6

Core Features 6

Conclusion 8

Introduction

Our online learning platform (Learnify) focuses on learner-oriented based teaching by collecting both teachers and students in a centralized course programme. Under development phase, our team utilized object-oriented principles, built strong team chemistry, and enhanced our basic understanding of CRUD (Create, Read, Update, Delete), Spring Data JPA (Jakarta Persistence API), Spring Boot, H2 Database, and Thymeleaf.

By integrating these robust technologies, we successfully engineered a secure, scalable MVC application that bridges the gap between educational content creation and interactive learning. The resulting platform empowers educators to design comprehensive courses and automated assessments, while providing learners with an intuitive dashboard to track their enrollment, attempt quizzes, and monitor their academic progress. Ultimately, this report details the architectural decisions, object-oriented domain modeling, and relational database strategies that form the backbone of Learnify, highlighting both our technical implementations and the problem-solving methodologies we adopted along the way.

Architectural Design

This project follows the MVC (Model-View-Controller) style production structure. This architecture is used for its increased maintainability, parallel and structured development, and security.

1. The Model

It holds the data structure of the class files, enforces rules, and talks with H2 database. To put it another words, it converts my class(@Entity classes) files into SQL data with the use of tags. Lives in "***src/main/java/com/platform/online_course/mode***l" directory.

1. **Examples**: `Course`, `Quiz`, `Question`, `Learner`, `Teacher`, `QuizAttempt`, `Review`.
2. **Entities are annotated with JPA annotations** like `@Entity`, `@OneToMany`, `@ManyToMany`, `@ManyToOne`, and inheritance via `BaseUser`.

2. The View

This is basically the frontend of our project. Thymeleaf (Java template engine) and Bootstrap 5(frontend framework) are utilized for rendering the UI. This layer fetches the data that is injected with Model attributes of the Controller layer, and with the use of

dynamic rendering, form binding, and route-safe links it gets displayed properly**.** These templates in `src/main/resources/templates` render dynamic pages.

1. **Dynamic rendering:**
   - `th:text`**,** `th:if`**,** `th:each` are used across dashboards and detail pages.
   - Example: teacher dashboard iterates `teacherCourses` and renders per-course actions.
2. **Form binding:**
   - `th:object` and `th:field` bind forms directly to Java objects **(**`Course`**,** `Quiz`**,** `Question`**).**
   - Examples: `create-course.html`**,** `quiz-create.html`**,** `quiz-edit.html`**,** `question-create.html`**.**
3. **Route-safe links/actions:**
   - `th:href` and `th:action` build parameterized URLs for course/quiz/question operations.

3. The Controller

This layer acts as a middle layer between model and view layers. With the utilization of mainly **@GetMapping** and **@PostMapping,** it handles routing, sessions, role checks, and data binding:

1. **Routing and page flow:**
   `AuthController` manages signup/login/logout.
   `DashboardController` serves learner/teacher dashboard flows and enrollment/review interactions.
   `TeacherController` handles teacher CRUD for courses/quizzes/questions
   `CourseController` handles learner quiz-taking flow (start, submit, result).
2. **HTTP session handling:**
   Unified session keys are used after login:
   `CurrUserID`
   `CurrUserRole`
   Most protected endpoints check for a valid session and role before proceeding.
3. **Data binding:**
   `@ModelAttribute` binds form payloads to entities (e.g., `Course`, `Quiz`, `Question`).
   Path variables (`@PathVariable`) are used for resource-scoped actions like `/course/{courseId}/quiz/{quizId}/...`**.**

Controller layer also handles multiple user types, and prevents authentication flow among various users and user types:

1. **Signup** in `AuthController` creates either `Learner` or `Teacher` based on role selection.

2. **Login validates credentials** and stores `CurrUserID` + `CurrUserRole` in session.

3. **Controllers enforce role-based redirects:**
   Learner-only routes redirect teachers away.
   Teacher-only routes redirect learners away.

This creates one authentication entry-point with role-specific authorization behavior downstream.

4. The Repository

The project currently has minimal dedicated service classes, and most use-cases are controller-driven with repository calls. Even with that service and repository layers are crucial for accessing the saved H2 database data:

1. **Dependency Injection**
   a. Repositories are constructor-injected into controllers.
   b. This keeps components testable and keeps us from manual implementation of each of them.
2. **Spring Data JPA**
   a. Repositories extend `JpaRepository` (e.g., `CourseRepository`, `TeacherRepository`, `LearnerRepository`, `QuizRepository`, `QuizAttemptRepository`, `QuestionRepository`, etc.).
   b. CRUD and other derived queries are used without any raw SQL.

As a result of these implementations database operations are handled through repository abstraction and JPA mapping, instead of direct SQL queries.

Object-Oriented Domain Model

This is the main section for setting up the proper foundation of the backend logic. Thus, inheritance hierarchy, polymorphism, and interfaces should be explained in detail, especially in a complex structured project like ours.

1. **BaseUser** abstract class: This parent class unites all the Learner and Teacher class objects. As we planned in pre-development stage, Teacher users will have the ability to create, and Learner users could enroll in the courses, we needed to separate them in a logical way. Which inheritance was the best fitting solution. @Inheritance(strategy = InheritanceType.JOINED) is used to tell the database this will be inherited by other classes, so that inheritance hierarchy can be mapped to a

normalized database schema. By making BaseUser abstract, we ensure the system cannot mistakenly instantiate a generic user without a defined role.

2. **Polymorphism and Dynamic Routing:** Beyond structural inheritance, the project leverages polymorphic behavior to dramatically simplify our web controller logic. By defining abstract methods such as getDashboardURL() and getRole() inside the BaseUser class, the application delegates routing decisions directly to the objects themselves. For instance, during the unified authentication process, the login controller does not need to use rigid if/else statements to check a user's type. It simply calls foundUser.getDashboardURL(), and the Java runtime dynamically dispatches the correct endpoint (/learner/dashboard or /teacher/dashboard) based on the specific instantiated object. This adheres strictly to the Single Responsibility Principle, keeping the controller clean and the routing logic encapsulated within the model.

3. **Interfaces and Behavioral Segregation:** To manage the complex, intersecting interactions between users, courses, and assessments, specific behaviors were abstracted into interfaces (specifically Enrollable, Reviewable, and Completable). Utilizing interfaces allows unrelated classes to share guaranteed behaviors without forcing them into an unnatural class hierarchy. A prime architectural example of this is the implementation of the Completable interface. Instead of placing the "completion" state on the Quiz class, which acts as a shared template for all students, the interface is implemented within the QuizAttempt association class. This acts as a unique "receipt" for the student, guaranteeing thread-safety, preventing state-leakage between concurrent users, and demonstrating a highly scalable approach to Object-Oriented state management.

Moreover, one of the most significant architectural challenges we faced during development was managing the state of a quiz while multiple users interacted with it simultaneously. Initially, it seemed logical to store a student's submitted answers directly within the Quiz entity. However, this approach presented a critical flaw. It violated thread-safety. As the Quiz class is universal, saving one Learner's data may not show any problem, but if 2 or more gets saved at the meantime this will result in overriding one's data with another.

We found the solution to be QuizAttempt Association class**.** To overcome this concurrency challenge, we engineered an **Association Class** named QuizAttempt. Rather than directly linking a Learner to a Quiz via a simple junction table, the QuizAttempt acts as an

independent transactional entity, much like a receipt generated at a store. It holds a **Many-to-One** relationship with both the Learner and the Quiz, effectively bridging them while isolating the user's specific state. QuizAttempt is uniquely responsible for storing the student's calculated score, completion status, and an @ElementCollection map linking specific Question IDs to the user's submitted text. By extracting this state into the QuizAttempt class, the Quiz remains a pure, stateless template. This separation of concerns strictly adheres to **SOLID** principles and ensures that the platform's grading algorithm remains perfectly thread-safe and scalable.

Database & Entity Relations

Even though raw SQL code is not used during the development of this project, database queries and entity relations are handled well-structured with the Spring Data JPA:

1. **@Entity:** is used to tell the database this class will be mapped into a database table.

2. **@ElementCollection:** as SQL tables do not have a direct implementation of complex data types, this key is for handling that. Basically, it creates a one-to-many relationship and maps the collection of basic types in a new table.

3. **@OneToMany(cascade = CASCADE.ALL):** like a @ElementCollection; however, this is for mapping collection of complex types. CASCADE.ALL means all the operation performed to parent will also be performed on the child entity.

4. **@GeneratedValue(strategy = GenerationType.IDENTITY):** mainly used with the **@Id** key, it tells database that how will this variable will get generated. GenerationType.IDENTITY means id will keeps increasing every time new data is generated (e.g. ID=1, 2, 3, ...).

5. **@JoinColumn(name = "something"):** Specifies the foreign key column in the database table of the current entity. Mainly used with the type of relation type of **@ManyToOne.**

Core Features

1. **Unified Authentication**

A single login/signup flow supports both learners and teachers.

- Signup chooses role and creates role-specific user type.

- Login stores session identity + role.

- Logout invalidates session.

- Protected routes rely on session attributes to authorize access.

This centralizes authentication while allowing role-dependent behavior in controllers.

2. Role-Based Access Control (RBAC)

RBAC is implemented manually at controller-entry level using session checks.

Typical guard pattern:

- If `CurrUserID` missing -> redirect to `/login`

- If role mismatch -> redirect to the proper dashboard (`/learner/dashboard` or `/teacher/dashboard`)

Additional ownership checks are used in teacher quiz/question management routes:

- Teacher must own the target course (`teacher.getCreatedCourseIDs().contains(courseId)`) before editing quiz data.

Practical outcome:

- Learners are blocked from teacher management routes.

- Teachers are blocked from learner quiz-taking routes.

- Course/quiz management is narrowed to owned resources in relevant endpoints.

3. Stateless Quiz Grading (`calculateScore()`)

The grading logic is template-driven and map-based:

- `Quiz.calculateScore(Map<Long, String> answerMap)` grades by comparing submitted answers with correct answers.

- The method computes score using quiz question set + max score, without depending on mutable global state.

- `CourseController.submitQuiz(...)` normalizes request data into a map keyed by question id, then passes it into `calculateScore()`.

Why this is stateless and scalable:

- The quiz is used as a grading template (question/answer definition).

- Submitted answers are provided as an input map for each request.

- No shared mutable grading state is required across users.

Conclusion

Ultimately, the development of this online course platform successfully demonstrated the practical application of advanced Object-Oriented Programming principles within a modern Spring Boot MVC architecture. By meticulously planning the domain model—leveraging inheritance for distinct user roles and interfaces for shared behavioral states—we established a highly scalable and maintainable backend foundation. Furthermore, diagnosing and resolving concurrency challenges through the implementation of the QuizAttempt association class highlighted the critical importance of designing a properly normalized, thread-safe database schema.

This project was not merely an exercise in writing functional syntax, but a comprehensive deep dive into real-world software design. It effectively bridged the gap between abstract OOP concepts and tangible, secure web development, resulting in a robust educational platform built on professional engineering standards.